DistilBERT Question-Answering Pipeline

A Deep-Dive Industrial Overview

`distilbert-base-cased-distilled-squad` | Extractive QA | HuggingFace Transformers

Architecture	Parameters	Fine-tuned on	Output
DistilBERT 6L	66 Million	SQuAD v1.1	Answer + Score

Agenda

01 Model Architecture — DistilBERT layers, heads, distillation

02 Tokenisation Flow — Input encoding & token mapping

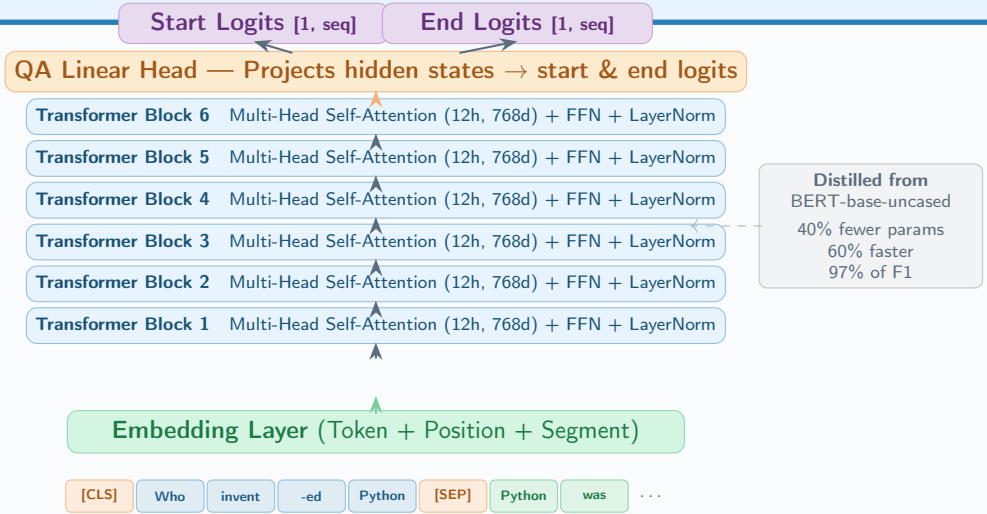
03 Inference Pipeline — Manual QA function design

04 Answer Extraction — Logit decoding & span selection

05 Score Computation — Confidence from softmax

06 Full Code Walkthrough — Annotated source listing

01 — DistilBERT Architecture



02 — Tokenisation Flow

Question: *"Who invented Python?"*

Context: *"Python was created by Guido. . . "*

AutoTokenizer (WordPiece, max_length=512, truncation=True, return_tensors="pt")

Token IDs:

[CLS]	Who	in-	##ven	##ted	Python	?	[SEP]	Python	was	···

Segment:

A	A	A	A	A	A	A	A	B	B	B

A = Question tokens **B** = Context tokens **[CLS]/[SEP]** = Special tokens

input_ids [1,L]

attention_mask [1,L]

token_type_ids [1,L]

03 — Inference Pipeline Design

question (str) + context (str)

↓

AutoTokenizer.__call__()

truncation=True, max_length=512
Pair encoding: [CLS] Q [SEP] Context [SEP]
Returns pt tensors

↓

.to(DEVICE) — GPU/CPU placement

↓

torch.no_grad() + _model(**inputs

eval() = no dropout
no_grad() = skip gradient tape
2–3× faster, less memory

↓

start_logits & end_logits

Shape [1, seq_len] each
One logit score per token position

↓

argmax → slice → decode string

argmax gives token indices
convert_ids_to_tokens → string

↓

{"answer": str, "score": float}

04 — Answer Span Extraction

Start Logits

End Logits

Token Sequence with Selected Span

[CLS]	Who	invented	[SEP]	Python	Guido	van	Rossum	was	···
0	1	2	3	4	5	6	7	8	

$\text{tokens}[5:6+1] = \text{“Guido van”}$

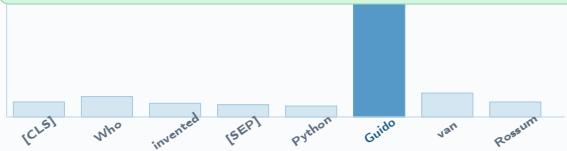
argmax=5

argmax=6

score

score

convert_ids_to_tokens → convert_tokens_to_string → "Guido van Rossum"



05 — Confidence Score Computation

Confidence Score Formula

$$s = \underbrace{\mathrm{softmax}(\mathrm{s}_{\mathrm{start}})[\hat{i}]}_{\text{start prob}} \times \underbrace{\mathrm{softmax}(\mathrm{s}_{\mathrm{end}})[\hat{j}]}_{\text{end prob}}$$

Step 1: `start_logits.softmax(-1)` → prob distribution

Step 2: `end_logits.softmax(-1)` → prob distribution

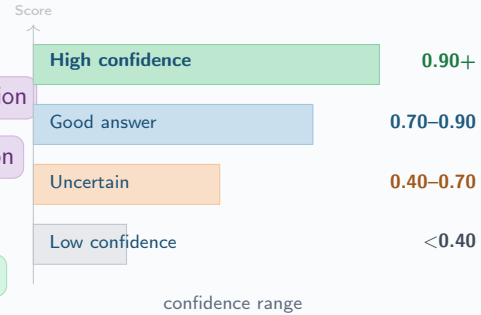
Step 3: Index at argmax positions → scalar probs

Step 4: Multiply → final confidence `score: float`

```
{"answer":"Guido van Rossum","score":0.9731}
```

Example output

Score Interpretation



Product of two independent softmaxes. True joint scoring uses a 2-D score matrix, but this approach

06 — Full Code Walkthrough

distilbert_qa.py

```python
from transformers import (
    AutoTokenizer,
    AutoModelForQuestionAnswering
)
import torch

MODEL = "distilbert-base-cased-distilled-squad"

# Load once -- reuse many times
_tok   = AutoTokenizer.from_pretrained(MODEL)
_model = AutoModelForQuestionAnswering\
            .from_pretrained(MODEL).to(DEVICE)
_model.eval()       # disable dropout

def qa_distilbert(question: str,
                  context: str) -> dict:
```

Step 1 — Tokenise

Pair encoding: Q + [SEP] + Context. WordPiece handles OOV. Tensors placed on DEVICE.

Step 2 — Forward Pass

`eval()` disables dropout. `no_grad()` skips gradient tape. 2–3× faster inference.

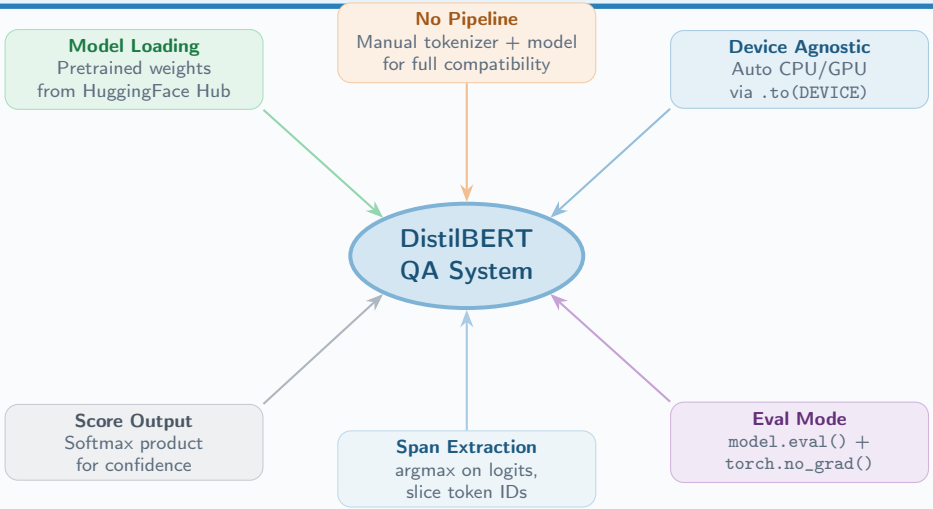
Step 3 — Span Decode

argmax on logits gives token indices. Slice the ID list and convert to human-readable text.

Step 4 — Score

Softmax → index at argmax → scalar. Product of start × end probability.

System Summary & Key Design Choices



Drop-in replacement for the removed HuggingFace `question-answering` pipeline — identical call signature, full control over internals.

Thank You

DistilBERT QA Pipeline — Industrial Deep Dive

66M
Parameters

6
Transformer Layers

SQuAD
Fine-tune Dataset